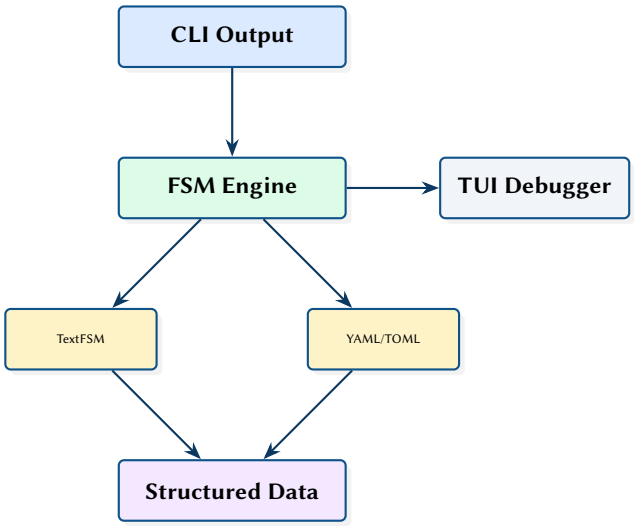
cliscrape: A High-Performance CLI Parsing Engine with Interactive TUI Debugger

TextFSM-Compatible State Machine Parser in Rust

Simon Knight

March 2026 • Version 0.1.0

Last updated: March 12, 2026



Abstract. Network automation pipelines rely heavily on parsing unstructured CLI output from network devices. This report presents cliscrape, a Rust implementation of a TextFSM-compatible state machine parser that provides 10–50× performance improvement over the Python reference implementation while maintaining full template compatibility with the existing NTC-Templates library. The system includes a modern YAML/TOML template format as an alternative to the TextFSM DSL, and a TUI debugger for real-time template development with FSM state visualisation and regex match highlighting.

Contents

1 Introduction and Motivation

The networking industry relies on CLI-based management for device configuration and monitoring. Parsing this unstructured text output is often the bottleneck in automation pipelines. TextFSM [1] provides a template-based approach using state machines, and the NTC-Templates [2] library offers hundreds of community-maintained templates.

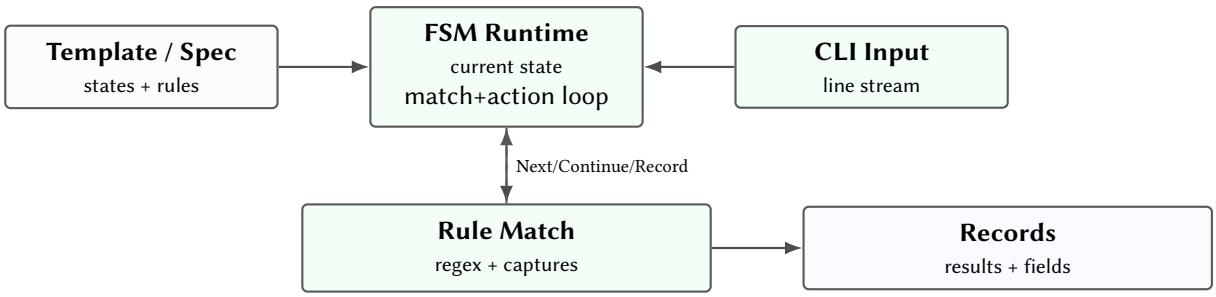
However, the Python TextFSM implementation has three limitations:

1. **Performance** — Python's interpreted execution and GIL make it 10–50× slower than compiled alternatives, particularly for large `show tech-support` outputs.
2. **Readability** — The TextFSM DSL uses inline regex with positional variable references, producing "regex soup" that is difficult to debug.
3. **Observability** — When a template fails to parse correctly, there is no tooling to visualise state transitions and identify which rule matched (or failed to match) each line.

: Full TextFSM Compatibility

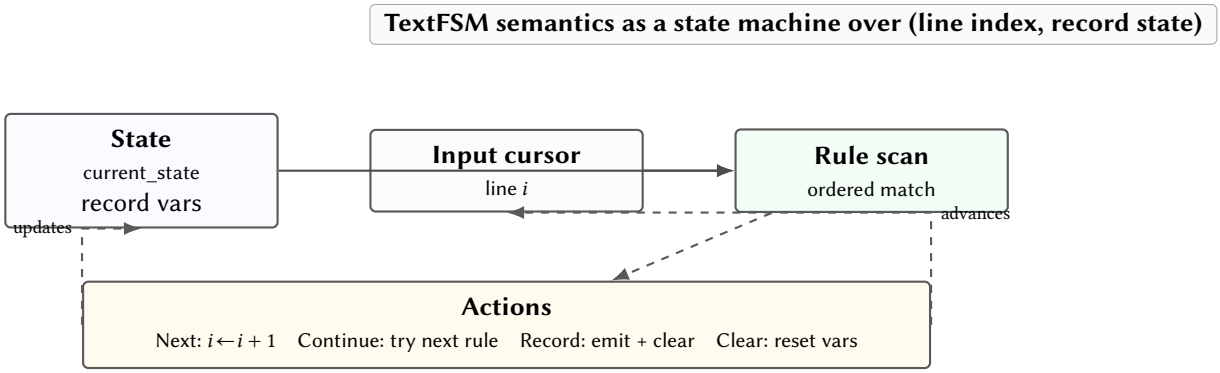
clicrape must parse every valid TextFSM template identically to the Python reference implementation. Existing NTC-Templates must work without modification, enabling zero-cost migration.

2 FSM Engine



Control vs data. The same regex match both extracts data (captures) and drives control (state transitions). This is what makes TextFSM robust to prompts, pagination, and error banners.

Figure 1. FSM engine model: rule matching is both evidence extraction and control-flow.



Conceptual point. TextFSM is effectively a small interpreter: the parse result depends on both the current state and the record accumulator. Making this explicit explains why ordering matters and why a single regex match can both extract data and change control flow.

Figure 2. TextFSM parsing as an interpreter over a line cursor and an accumulating record. This view is useful for debugging template failures and performance hotspots.

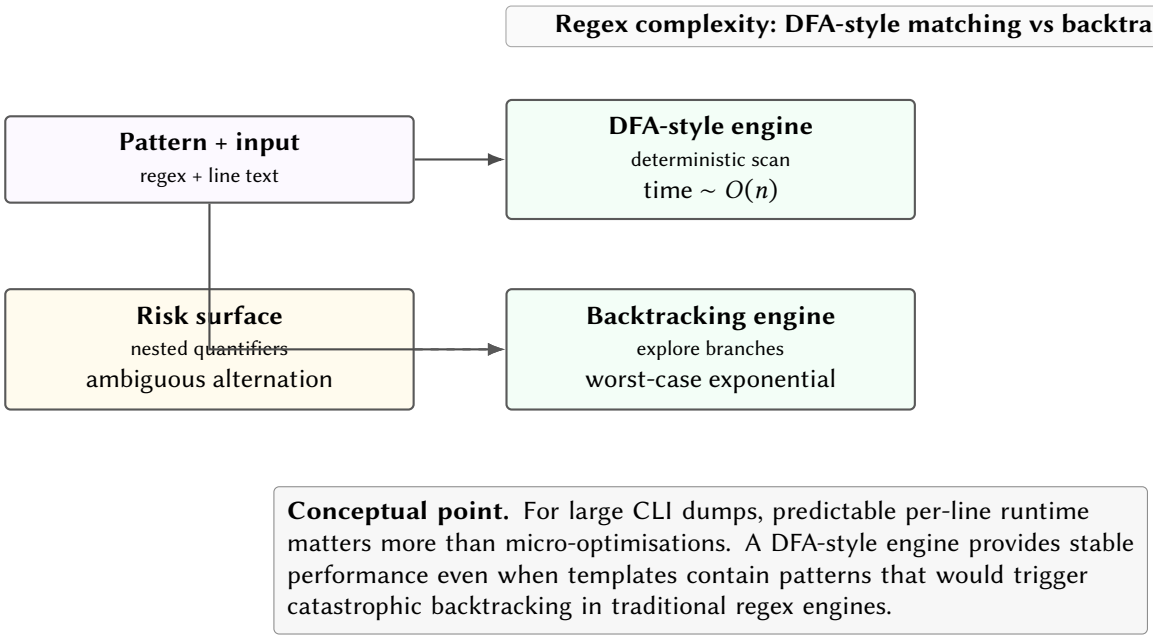
The core engine implements the TextFSM state machine semantics:

```rust
pub struct FsmEngine {
    values: Vec<ValueDef>,
    states: HashMap<String, Vec<Rule>>,
    current_state: String,
    record: HashMap<String, String>,
    results: Vec<HashMap<String, String>>,
}

pub struct Rule {
    pub pattern: Regex,
    pub action: Action,       // Next, Continue, Record, Clear
    pub next_state: Option<String>,
    pub captures: Vec<String>, // Variable names to capture
}
```

For each input line, the engine tries rules in the current state sequentially. On match, captured groups are assigned to named variables, and the specified action is executed:

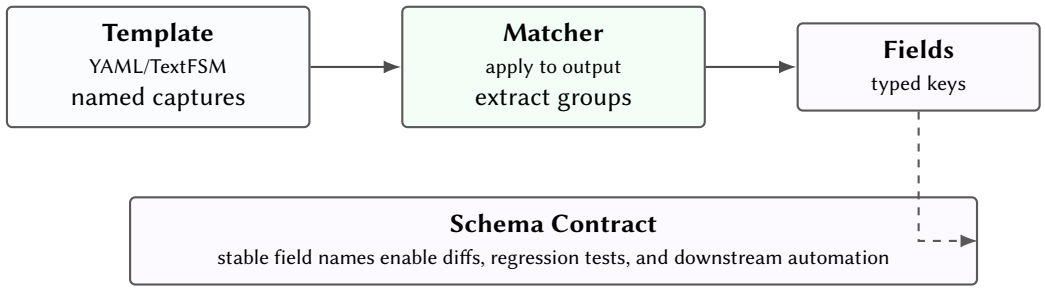
- **Next** — advance to next input line
- **Continue** — stay on current line, try next rule
- **Record** — save current record, start new one
- **Clear** — reset all variables to defaults



…DFA-based engine provides predictable O(n) matching per line, avoiding the catastrophic backtracking possible with Python's re module on pathological patterns.

Figure 3. Why the regex engine choice matters. DFA-style matching keeps runtime predictable on long inputs, while backtracking engines can blow up on certain pattern classes.

3 Template Formats



Failure mode. If device firmware changes table formatting, the template breaks visibly and tests fail, rather than silently producing incorrect structured output.

Figure 4. Templates define the parsing contract: semistructured CLI output becomes stable, typed records.

3.1 TextFSM Compatibility

A dedicated parser reads `.textfsm` files and translates them into the internal Finite State Machine (FSM) representation. This handles the full TextFSM grammar including value options (`Required`, `List`, `Filldown`, `Fillup`, `Key`).

3.2 Modern YAML Format

An alternative YAML format reduces regex complexity:

```
values:
  interface: \S+
  ip_address: \d+\.\d+\.\d+\.\d+|unassigned
  status: up|down|administratively down
  protocol: up|down

states:
  Start:
    - match: ^${interface}\s+${ip_address}\s+\S+\s+\S+\s+${status}\s+${protocol}
      action: Record
```

Design Decision 1: Dual Template Format

Supporting both TextFSM and YAML formats allows teams to migrate incrementally. Existing templates work immediately; new templates can use the more readable YAML format. Both compile to the same internal FSM representation.

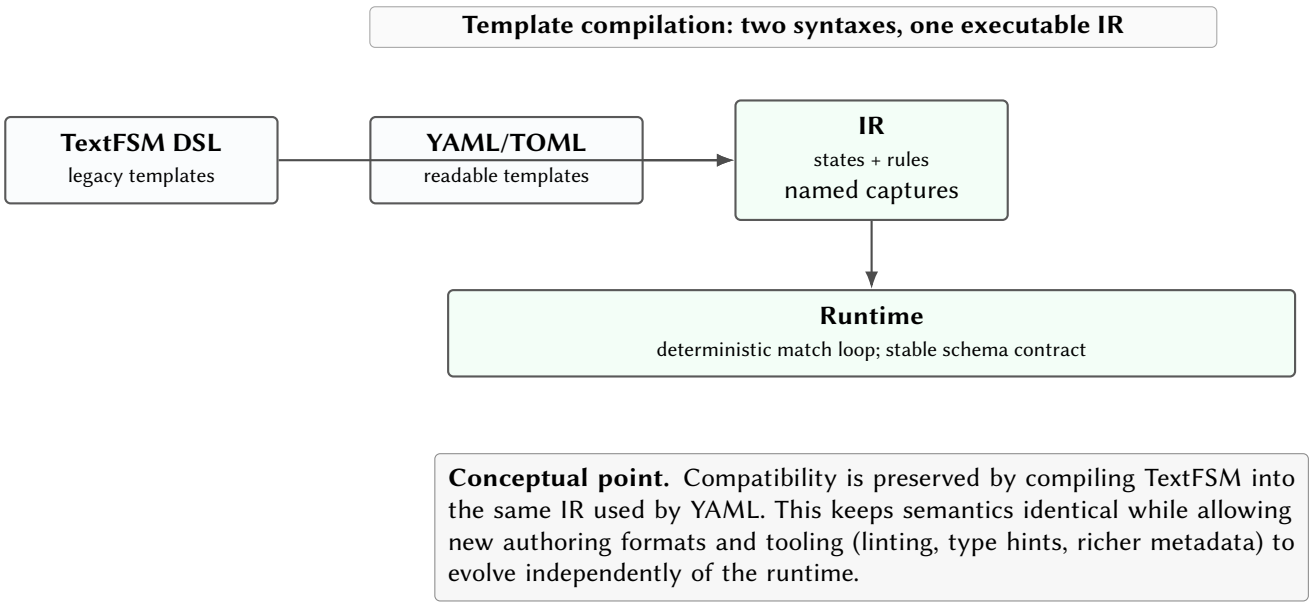


Figure 5. Compilation boundary: both template syntaxes compile into one IR consumed by the same runtime, ensuring identical semantics and enabling consistent debugging tools.

4 TUI Debugger

The TUI debugger [3] provides four panes for interactive template development:

1. **Input Pane** — Raw CLI output with line highlighting showing which line is currently being processed.
2. **State Pane** — Current FSM state with transition history, showing which rule triggered each transition.
3. **Variables Pane** — Real-time display of all defined variables and their current values.
4. **Diff View** — Regex match highlighting showing exactly which portion of each line matched each capture group.

The debugger supports step-by-step execution, breakpoints on specific states or line numbers, and "run to record" mode that advances to the next Record action.

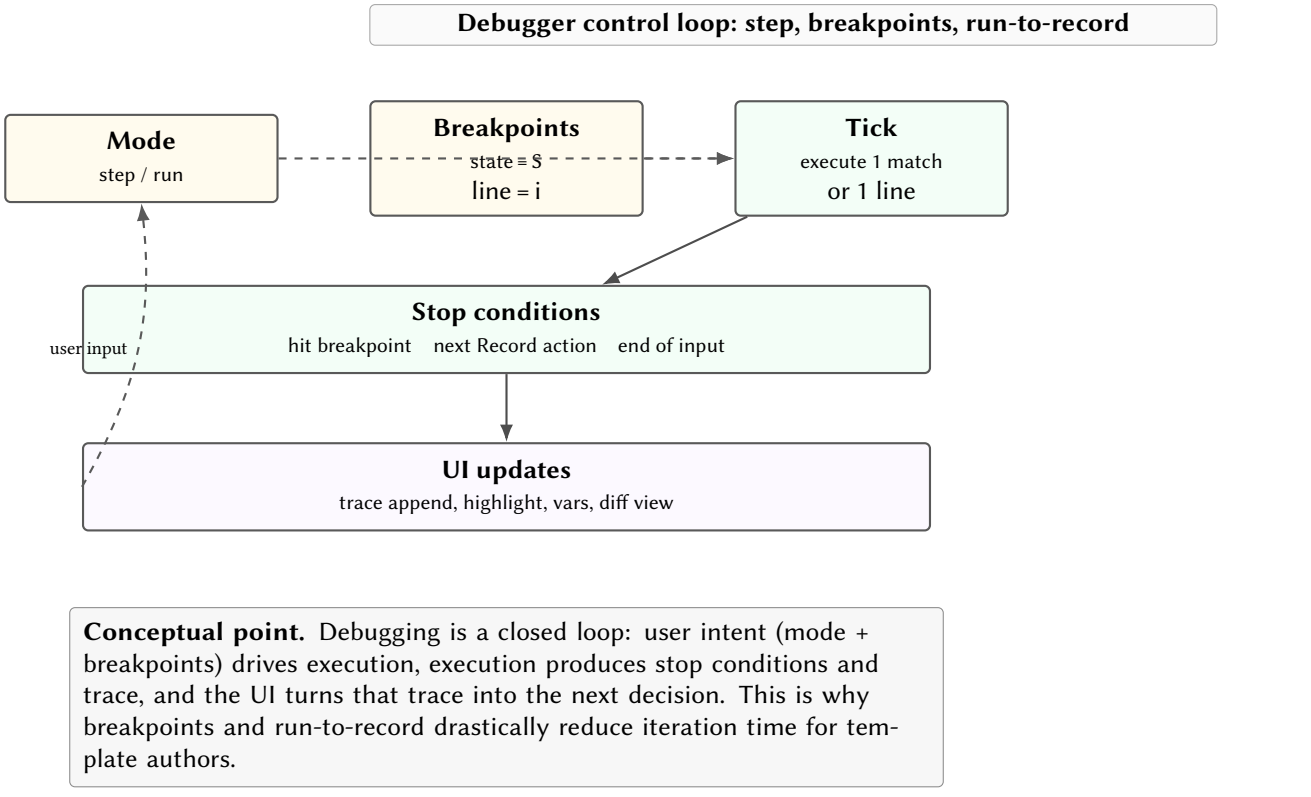


Figure 6. Debugger as a control loop over the parsing interpreter. The same trace that explains failures also drives interactive navigation (step/run/breakpoints).

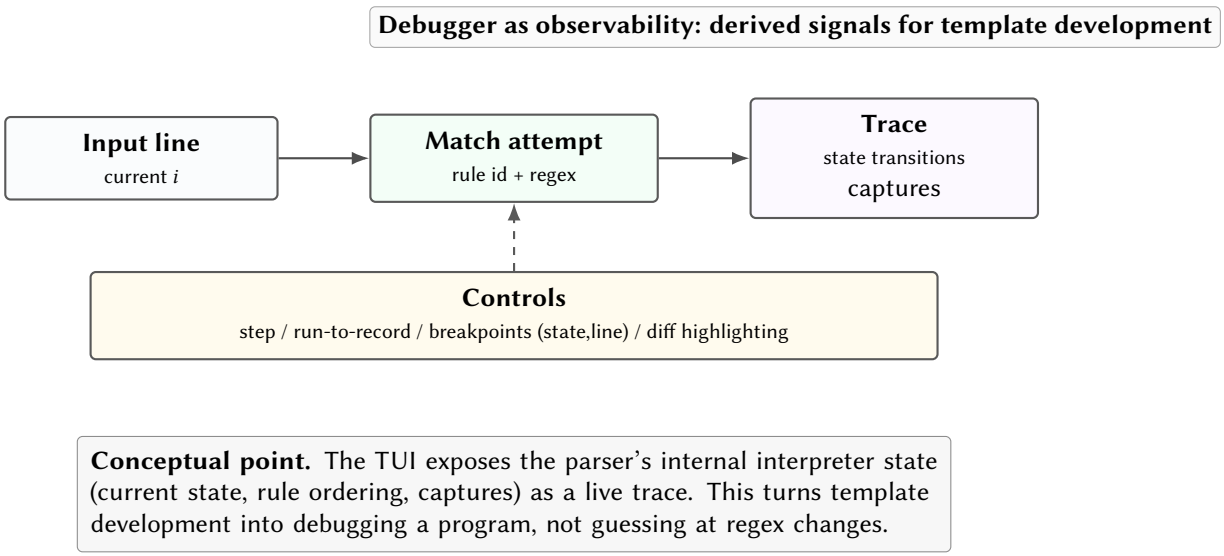


Figure 7. TUI debugger as observability for the parsing interpreter. Interactive controls operate on a trace of state transitions and captures.

5 Design Summary

Table 1. Key design decisions.

Decision	Rationale
Rust regex crate	DFA-based $O(n)$ matching; no backtracking vulnerabilities
Full TextFSM compat	Zero-cost migration from existing NTC-Templates
Dual template formats	Incremental adoption; YAML for new templates
TUI debugger	Real-time FSM visualisation eliminates trial-and-error template development

References

[1] Google, *TextFSM: Template-based state machine for parsing*, 2024. [Online]. Available: https://github.com/google/textfsm

[2] Network to Code, *NTC Templates: Textfsm templates for network devices*, 2024. [Online]. Available: https://github.com/networktocode/ntc-templates

[3] Ratatui contributors, *Ratatui: A Rust library for cooking up TUIs*, 2024. [Online]. Available: https://ratatui.rs

List of Figures

List of Tables

List of Design Decisions & Rules

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report